Compiladores paralelizadores

(continuação)

Análise de Dependências de Dados

- Identificar dependências de dados para detectar paralelismo implícito e otimizar programas sequenciais
  - Dependência Verdadeira (RAW - Read After Write)
    - Exemplo: A = B + C → D = A + 2 ← (execução em sequência, 1 e 2)
      - A instrução 2 depende do valor de A gerado na instrução 1
  - Antidependência (WAR - Write After Read)
    - Exemplo: A = B + C → B = D / 2
      - A instrução 1 usa o valor antigo de B antes de ser atualizado na 2
  - Dependência de Saída (WAW - Write After Write)
    - Exemplo: A = B + C → A = E + F
      - A ordem de execução afeta o valor final de A
  - Dependência de Controle:
    - Ocorre em desvios condicionais
      - Exemplo: if (X >= 0) then A = A + 2 ← Depende do valor de X para ser executado

Dependências em Laços de Repetição

- Dependência RAW

```
      do I = 2, N
(1)      A(I) = B(I) + C(I)
(2)      D(I) = A(I)
      end do
```

Se a Instrução 2 fosse executada antes da Instrução 1, ela leria um valor obsoleto ou incorreto de A(I)

- Dependência RAW entre iterações

```
      do I = 2, N
         A(I) = B(I) + C(I)
         D(I) = A(I-1)
      end do
```

← Valor de A da iteração anterior

- Dependência WAR (antidependência) entre iterações

```
do I = 2, N
   A(I) = B(I) + C(I)
   D(I) = A(I+1)
end do
```

- A iteração I deve obrigatoriamente completar sua leitura de A(I+1) antes que a iteração I+1 execute sua escrita em A
- Se a iteração I+1 fosse executada antes ou em paralelo sem sincronização, ela sobrescreveria o valor de A que a iteração I ainda precisa ler

Vetorização de Instruções

- Código original

```
do I = 1, N
   A(I) = B(I+2) + C(I+1)
end do
```

- Código vetorizado
  - A(1:N) = B(3:N+2) + C(2:N+1)

Compiladores vetorizadores convertem loops em instruções vetoriais

Paralelização de Instruções

- Distribuir iterações de loops entre processadores em arquiteturas multiprocessadas
  - Iterações Independentes. Paralelização possível no laço externo (I)

```
do I = 1, N
   do J = 2, N
      A(I,J) = B(I,J) + C(I,J)
      C(I,J) = D(I,J) / 2
      E(I,J) = A(I,J-1)**2 + E(I,J-1)
   end do
end do
```

(continua)

(continuação)

- Iterações Dependentes. Sincronização necessária devido à dependência entre iterações

```
do I = 2, N
   A(I) = B(I) + C(I)
   C(I) = D(I) * 2
   E(I) = C(I) + A(I-1)
end do
```

O laço de repetição não pode ser simplesmente dividido pois há uma dependência de dados

- Escalonamento em Blocos Contínuos
  - As iterações do loop são divididas em blocos contínuos, onde cada processador recebe um conjunto sequencial de iterações
- Auto-Escalonamento (Self-Scheduling)
  - As iterações são atribuídas dinamicamente aos processadores conforme eles ficam disponíveis

Linguagem Linda: Programação Paralela Baseada em Espaço de Tuplas

- Modelo de programação paralela baseado em espaço de tuplas
  - (“Linda is powerful”, 2, 32.5, 62)
  - (1,2, “Linda is efficient”, a:20)
  - (“Linda is easy to learn”, i, f(i))

Uma tupla é uma sequência de campos tipados

- Comunicação entre processos de forma assíncrona
- Pode ser integrada a várias linguagens (ex.: C, Fortran)
- Insere/remove/lê uma tupla do espaço compartilhado
- Foco em comunicação indireta e coordenação implícita
- Ideal para sistemas distribuídos e ambientes heterogêneos

Linguagem Java: Paralelismo Explícito com Threads e Concorrência

- Java oferece suporte nativo para programação paralela através de threads e bibliotecas de concorrência
- A linguagem é amplamente usada em sistemas distribuídos e multicore
- Possui recursos para threads, concorrência, e memória compartilhada
- Facilidade de uso com APIs bem documentadas
- Grande ecossistema e suporte para frameworks paralelos (ex.: Hadoop, Spark)

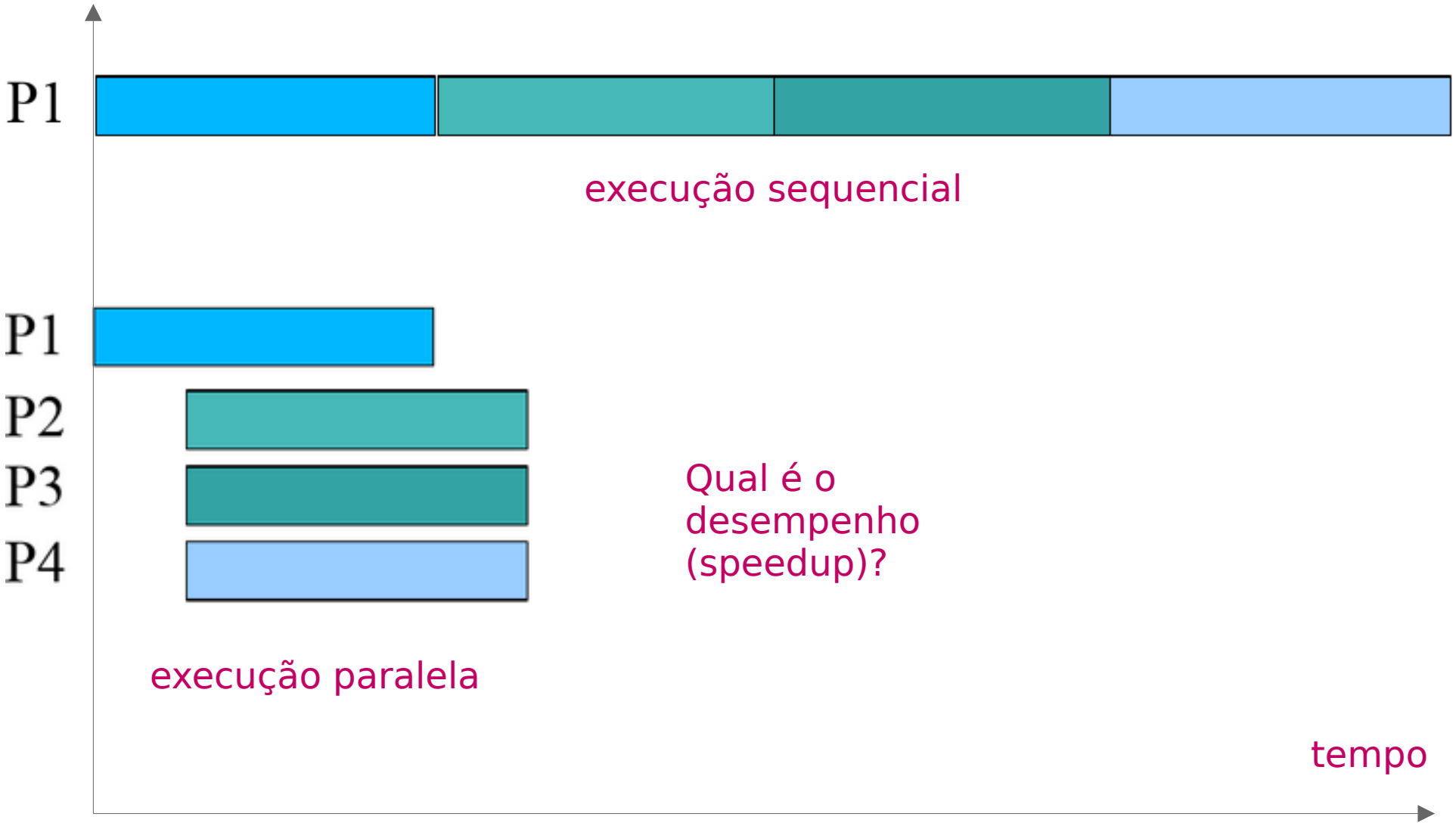
Linguagem HPF (High-Performance Fortran)

- Extensão da linguagem Fortran para programação paralela em sistemas de memória distribuída

- Principais Características
  - Diretivas de Paralelismo
    - Anotações no código (ex.: DISTRIBUTE, ALIGN) guiam o compilador na distribuição de dados e tarefas
  - Modelo de Dados Distribuídos
    - Arrays são divididos entre processadores para processamento paralelo
  - Foco em Granularidade Grossa
    - Adequado para problemas que envolvem grandes conjuntos de dado
  - Mantém a sintaxe familiar do Fortran
  - Alta performance em sistemas de memória distribuída

Desempenho

Execução sequencial X paralela

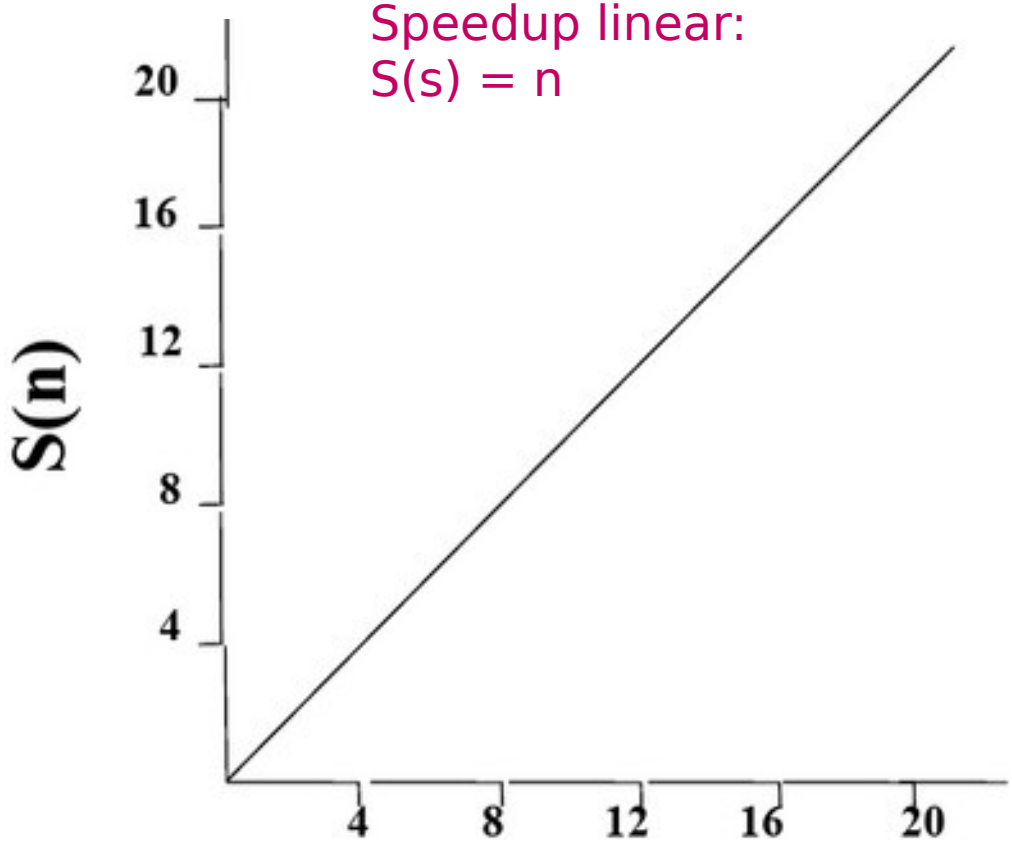


Speedup (fator de aceleração)

$$S(n) = \frac{\text{tempo de execução serial}}{\text{tempo de execução paralela}} = \frac{t_s}{t_p}$$

ideal para máquina paralela com n processadores/núcleos

Speedup linear:
S(s) = n



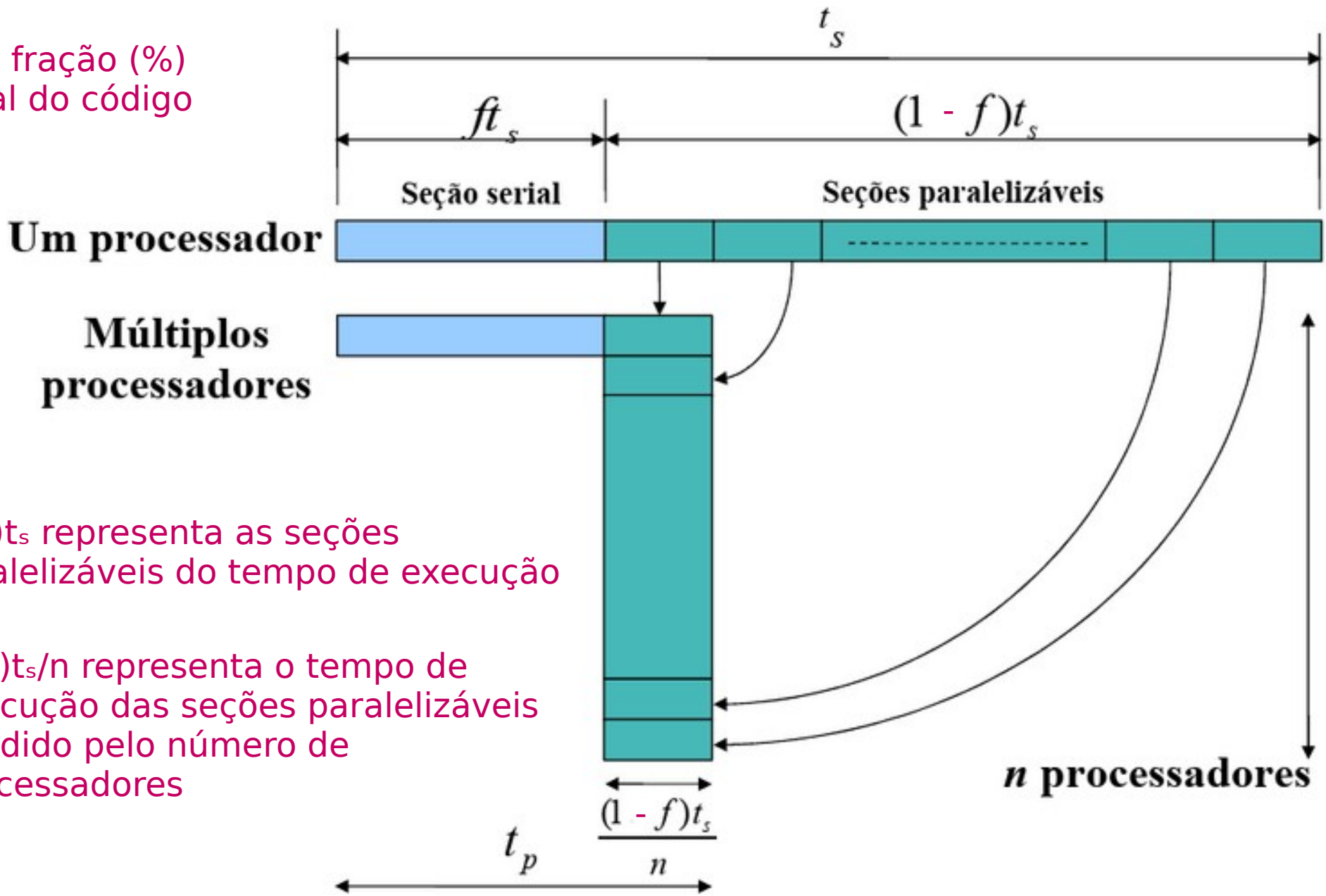
Speedup

- S(n) > n (superlinear)
  - O desempenho do programa paralelo é mais do que n vezes mais rápido do que o programa sequencial
  - O algoritmo sequencial original não é a maneira mais eficiente de resolver o problema
  - A arquitetura da máquina paralela possui características específicas, como memória extra ou um sistema de cache mais eficiente
    - Ex.: se cada processador em um sistema paralelo tiver sua própria memória, o conjunto total disponível pode ser maior do que em um sistema sequencial
      - Isso pode reduzir a necessidade de troca de dados entre a memória principal e a memória cache, resultando em um speedup > que o esperado

- S(n) < n (sub-ótimo)
  - Lei de Ahmdal
  - Sobrecarga do paralelismo (ex.: comunicação)

Speedup máximo

f é a fração (%) serial do código



$(1-f)t_s$ representa as seções paralelizáveis do tempo de execução

$(1-f)t_s/n$ representa o tempo de execução das seções paralelizáveis dividido pelo número de processadores

Eficiência

$$E = \frac{\text{tempo de execução serial}}{\text{t.exec.paralela} \times \text{nº de processadores}} = \frac{t_s}{t_p \times n}$$

$$E = \frac{S(n)}{n} \times 100\%$$

A eficiência fornece a fração de tempo que os processadores estão sendo utilizados

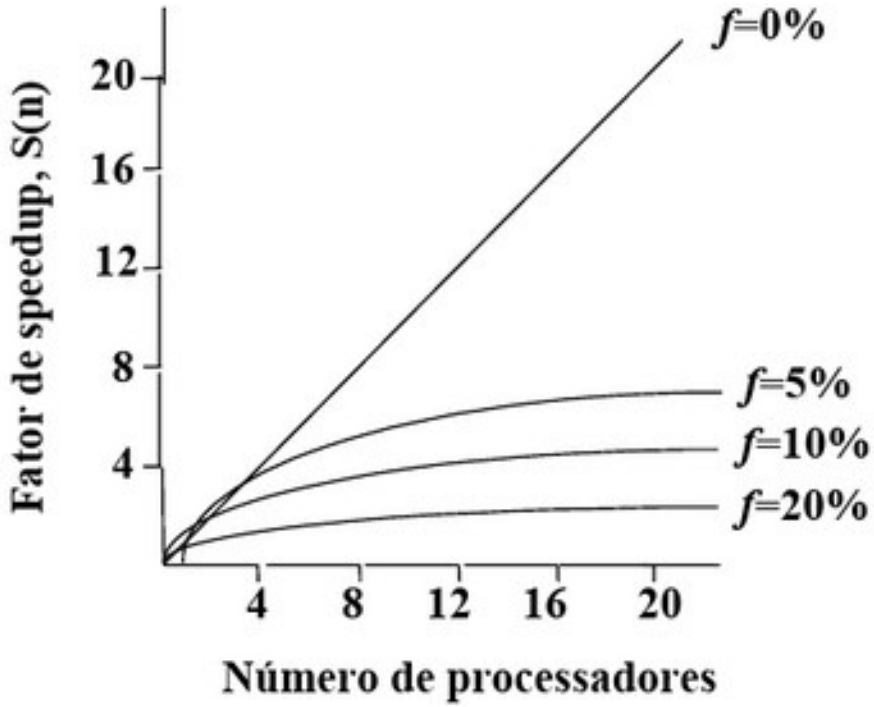
Lei de Amdahl (1967)

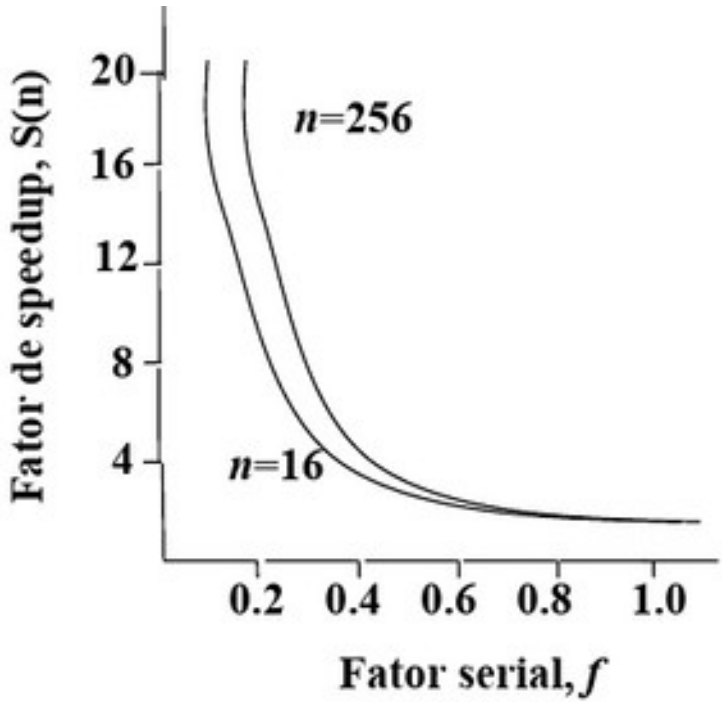
- O fator de speedup é dado por

$$S(n) = \frac{t_s}{f t_s + (1-f)\frac{t_s}{n}} = \frac{n}{1+(n-1)f}$$

- Speedup limitado pela fração serial
- Considera que o tamanho do problema é fixo
- Speedup máximo: $S_{max} = 1 / f$

Lei de Amdahl





- Mesmo com número infinito de processadores o speedup é limitado a 1/f
  - Ex.: para f = 5%, o speedup máximo é 1/0,05 = 20, independente de n

Lei de Gustafson (1988)

$$\text{Speedup} = f + P\,(1-f)$$

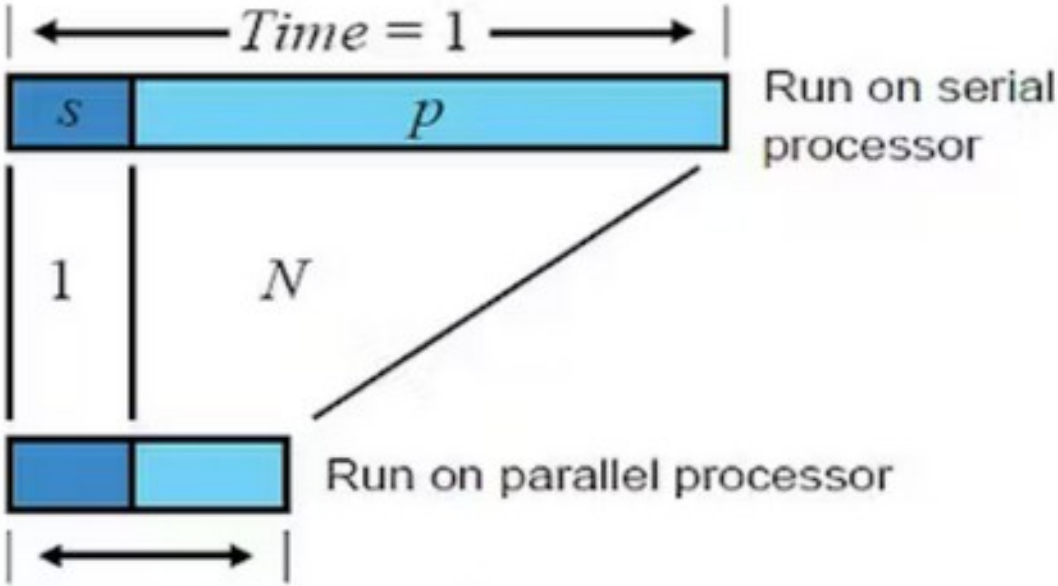
Onde:
f = fração serial do problema escalonado (a parte que não pode ser paralelizada)

(1-f) = fração paralela do problema escalonado

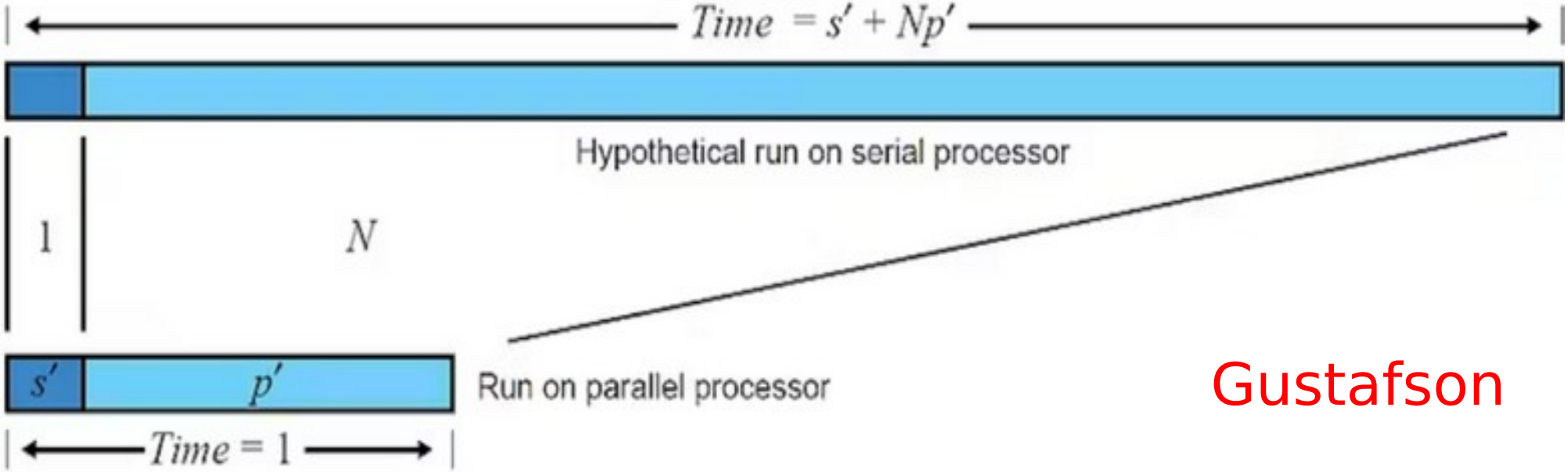
P = quantidade de processadores

- Reavalia a lei de Amdahl sob o ponto de vista da escalabilidade
- O tamanho do problema aumenta com o número de processadores
  - Pode-se resolver problemas maiores no mesmo intervalo de tempo
- Considera que o tempo de execução paralelo é fixo

Amdahl

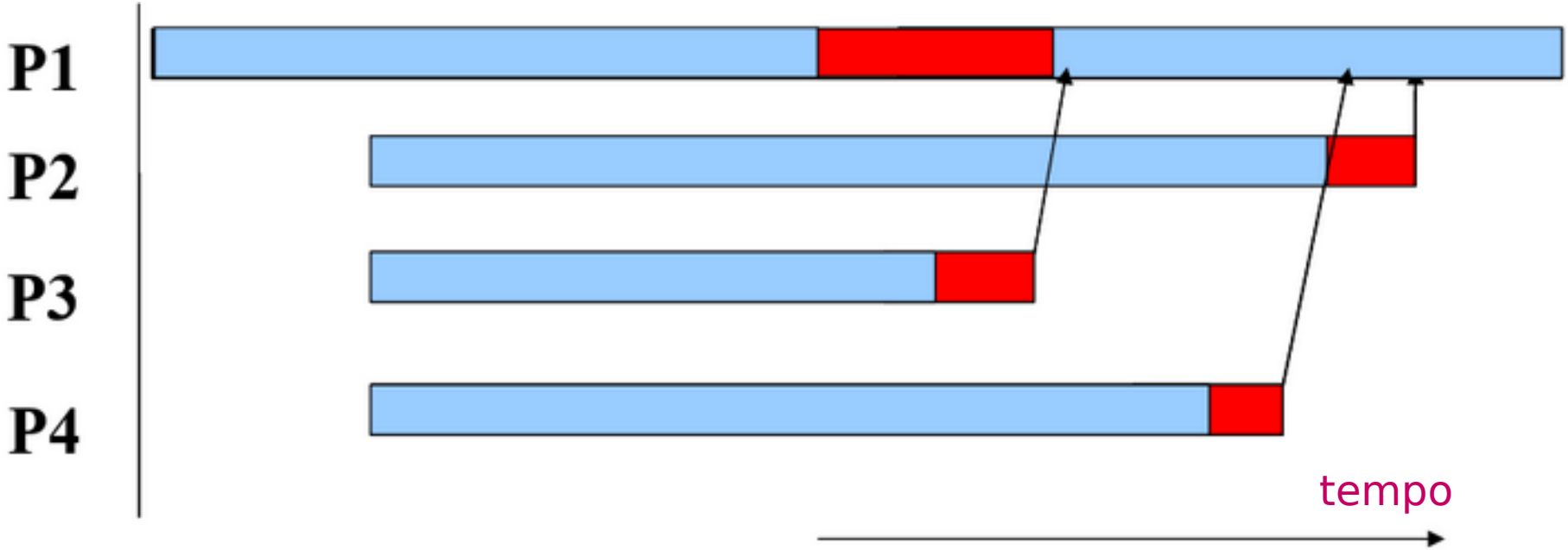


- s’ = parte serial
- p’ = parte paralela
- s’ = à fração serial (f)
- p’ = à fração paralela (1-f)
- N = quantidade de processadores



Gustafson

Sobrecargas na execução paralela



- Em vermelho, tempos gastos à mais, além do que foi gasto no cálculo sequencial
  - Ex.: ações ou esperas para comunicação, sincronização, criação de threads/processos
- Vão criando Overhead e reduzindo o Speedup

Escalabilidade

- Escalabilidade de hardware ou de arquitetura
  - Aumento do tamanho do sistema aumenta o desempenho
  - Facilidade de agregar processadores
- Escalabilidade do algoritmo paralelo
  - Problema de tamanho fixo
  - Problema de tamanho variável
- Escalabilidade do algoritmo
  - Algoritmo pode suportar um aumento do tamanho do problema com um aumento baixo e limitado dos passos de computação
    - Ex. 1: adição de matrizes - 2x o tamanho da matriz = 2x nro. de passos
    - Ex. 2: multiplicação de matrizes - 2x o tamanho da matriz = 4x passos

OpenMP

Criação de thread

- O #pragma omp parallel é usado para bifurcar threads
- A thread original será denotada como thread mestre com ID 0

```
#include <stdio.h>
#include <omp.h>

int main(void)
{
    #pragma omp parallel
    printf("Hello, world.\n");
    return 0;
}
```

- Compila: $ gcc -fopenmp hello.c -o hello -ldl
  - (o pacote gcc já inclui a biblioteca OpenMP)
- Executa: $ ./hello
- Saída:
  Hello, world.
  Hello, world.
- Pode ocorrer race condition:
  Hello, wHello, woorld.
  rld.

A função printf pode ou não ser atômica, dependendo da implementação

Exemplo

- Inicializar o valor de uma grande matriz em paralelo, usando cada thread para fazer parte do trabalho

```
int main(int argc, char **argv)
{
    int a[100000];

    #pragma omp parallel for
    for (int i = 0; i < 100000; i++) {
        a[i] = 2 * i;
    }

    return 0;
}
```

- Este exemplo é *“embaraçosamente paralelo”* e depende apenas do valor de i

- O sinalizador parallel for do OpenMP diz para dividir esta tarefa entre seus threads de trabalho
- Cada thread receberá uma versão única e privada da variável
- P. ex., com dois threads de trabalho, um thread pode receber uma versão de i que vai de 0 a 49999, enquanto o segundo recebe uma versão que vai de 50000 a 99999

Outro exemplo

alvo: Nvidia

Ex. 1
com #if

```
int v1[N], v2[N], v3[N];
#if defined(nvptx)
#pragma omp target teams distribute parallel for map(to:v1,v2)
map(from:v3)
for (int i= 0; i< N; i++)
    v3[i] = v1[i] * v2[i];
#else
#pragma omp target parallel for map(to:v1,v2) map(from:v3)
for (int i= 0; i< N; i++)
    v3[i] = v1[i] * v2[i];
#endif
```

Envia
Recebe

CPU

Ex. 2
sem #if

```
int v1[N], v2[N], v3[N];
#pragma omp target map(to:v1,v2) map(from:v3)
#pragma omp metadirective \
   when(device={arch(nvptx)}: \
         target teams distribute parallel for) \
   default(target parallel for)
for (int i= 0; i< N; i++)
    v3[i] = v1[i] * v2[i];
```

GPU

CPU

Exemplos de programas ou sistemas

Simulações Científicas: Previsão do Tempo

- Weather Research and Forecasting (WRF) Model
  - Modelo numérico de previsão do tempo amplamente utilizado que emprega paralelismo com MPI
  - https://www.mmm.ucar.edu/weather-research-and-forecasting-model
- Application of Parallel Processing - A Case Study on Weather Prediction
  - O artigo explora a aplicação do processamento paralelo em meteorologia preditiva, crucial para modelos climáticos e oceânicos
  - A modelagem numérica, que emprega algoritmos para antecipar as condições atmosféricas, depende do poder computacional do processamento paralelo para aumentar a precisão e a eficiência
  - https://acrg.uitm.edu.my/images/paper/v2_n1/pdf/A4_NSAB_12-15.pdf

Machine Learning: Treinamento de Redes Neurais Profundas

- Distributed Deep Learning with TensorFlow (Google)
  - https://www.tensorflow.org/guide/distributed_training
    - Treinamento distribuído usando paralelismo de dados e modelo
- PyTorch Distributed Training (Meta)
  - https://pytorch.org/tutorials/intermediate/ddp_tutorial.html
    - Treinamendo de redes neurais em paralelo
- Horovod: Distributed Deep Learning Framework (Uber)
  - https://horovod.ai/
    - Treinamento distribuído de redes neurais, compatível com TensorFlow, Keras, PyTorch e outros frameworks

Processamento de Imagens

- CUDA Code Samples
  - https://developer.nvidia.com/cuda-code-samples
    - O processamento usando GPU é um tipo de processamento paralelo usando milhares de núcleos menores e mais eficientes para realizar cálculos simultaneamente
- OpenCV e OpenMP: processamento paralelo para visão computacional
  - https://answers.opencv.org/question/103701/how-opencv-use-openmp-thread-to-get-performance/
    - OpenCV fornece as ferramentas para processamento de imagens e vídeos
    - OpenMP permite dividir o trabalho em várias threads, executadas simultaneamente em processadores multi-core